Sk. Abdul Kalam Azaad | HA-2 - FCND | 2400030294 S-2

1.) Kubernetes Deployment resource definitions by creating a deployment + service for any microservice and configure liveliness & readiness probes based on how the service behaves.

10

2.) Use resources limits and HPA by deploying a sample workload and configuring requests/limits and a basic HPA, then explain how HPA reacts when you generate load using kubectl or curlloop.

1.) Answer:

A Deployment manages a set of identical pods and ensures the desired number of replicas are always running.

It supports scaling, self-healing & rolling updates.

Microservices used are NGINX serving HTTP on port 80.

Deployment YAML

```
apiversion : apps/v1
Kind: Deployment
metadata :
    name : nginx-deplay
```

```
spec:
  replicas: 2
  selector:
    Match Labels:
      app: nginx
  template:
    metadata:
      Labels:
        app: nginx
    spec:
      Containers:
        - name: nginx
          image: nginx:latest
          Ports:
            - containerport: 80
```

Liveness probe checks whether the application is still running. If it fails, Kubernetes the container automatically.

Readiness Probe checks whether the application is ready to accept traffic. If it fails, traffic is stopped without restarting the Pod.

Both probes are configured using HTTP GET requests on the root (/) endpoint of the service.

A service (cluster IP) is created to expose the Deployment internally and route traffic and route traffic to healthy pods using label selectors.
Using probes improves availability. fault tolerance and prevents downtime by ensuring only healthy Pods receive traffic.

2.) Resource requests define the minimum CPU and memory guaranteed to a Pod, while limits restricts the maximum usage.
Proper resource configuration prevents a Pod from consuming excessive resources and ensure stable cluster performance.
A deployment is created with CPU requests (eg 100m) and limits (eg: 500m) to enable efficient scheduling

Horizontal Pod Autoscaler (HPA) automatically scales the number of Pods based on resource usage such as CPU utilization.

HPA requires the metrics-server to collect real-time CPU and memory metrics from Pods.

HPA is configured with minimum and max replicas and a target CPU utilization.

Load is generated using a kubectl busybox pod (or) curl loop. which increases CPU usage or the application.

When CPU usage exceeds the threshold, HPA scales Pods up when the load decreases, it automatically scales Pods down.